

Mastering Data Consistency Across Microservices



BYTEBYTEGO

FEB 27, 2025 • PAID



274



3



12

Share

Microservices architecture is a software design pattern where an application is built as a collection of small, independent services, each responsible for a specific function.

These services communicate with each other using APIs (Application Programming Interfaces) and operate independently, allowing for greater flexibility, scalability, and ease of maintenance. Think of a food delivery app with the following services:

- The order service manages customer orders.
- The payment service handles transactions.
- The restaurant service updates menu availability.
- The delivery service assigns and tracks deliveries.

Each service operates independently, allowing teams to update or scale them separately.

However, due to this separation, a major challenge with microservices is maintaining data consistency. In a monolithic system, all functionalities share a single database, resulting in consistent updates. On the other hand, microservices architecture advocates that each service should manage its database. While this is a good practice, it can lead to some scenarios such as:

- Duplicate or Lost Data
- Network Delays

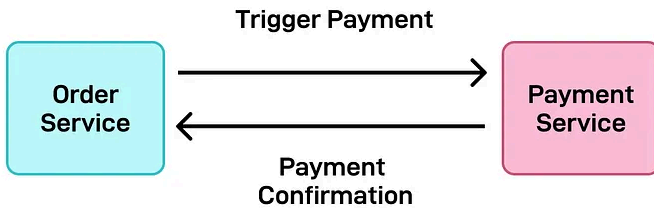
- Concurrency Issues

Understanding these scenarios is key to building robust, scalable applications using microservices. In this article, we will understand how data inconsistency can arise in microservices architecture and various strategies to deal with it.

Data Consistency Strategies For Microservices



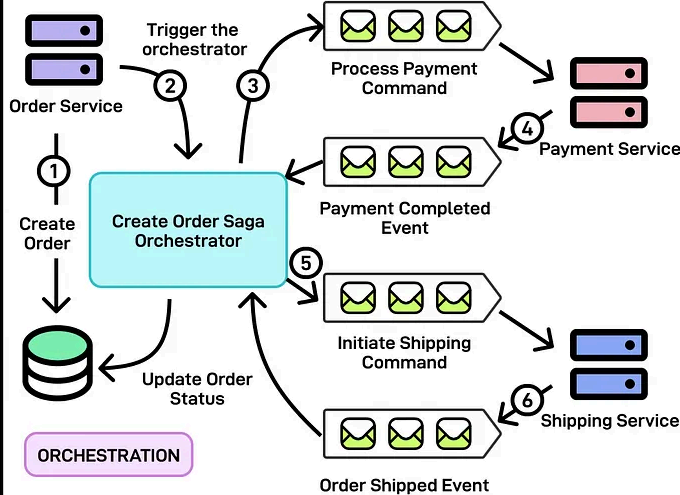
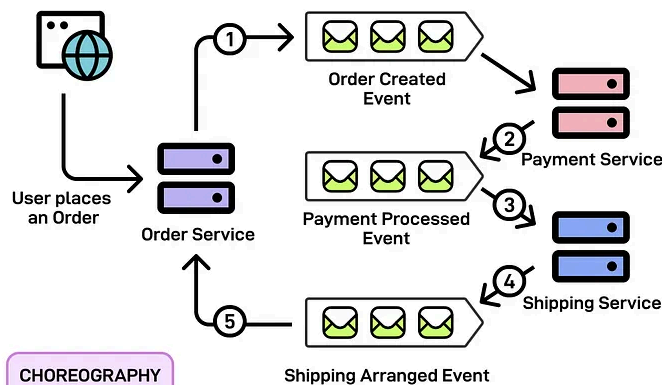
1 Synchronous Approach



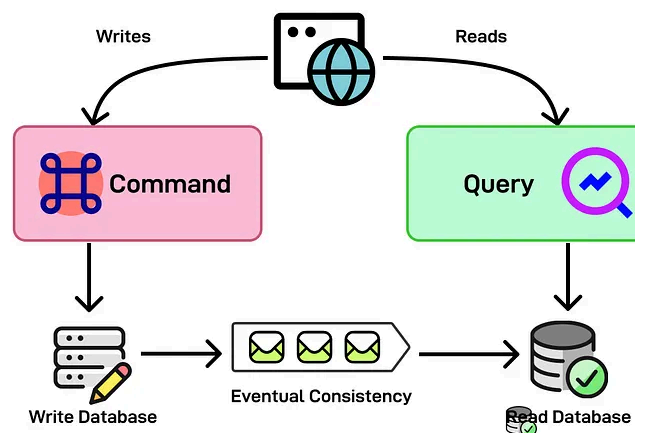
2 Asynchronous Approach



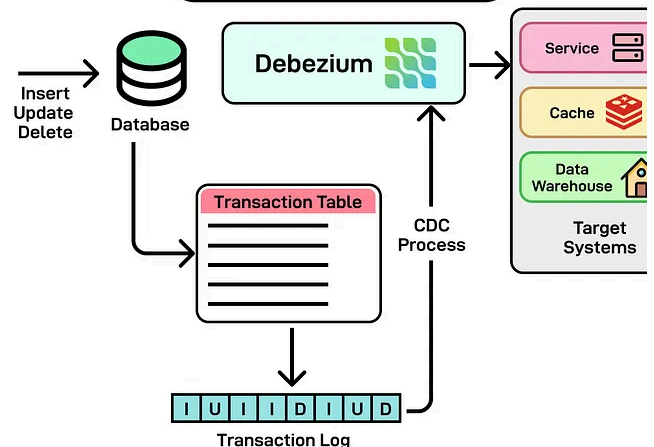
3. Choreography vs Orchestration



4. CQRS



5. Change Data Capture



Understanding Data Consistency in Microservice

Data consistency in microservices refers to the guarantee that all services in a system see the same data at the same time.

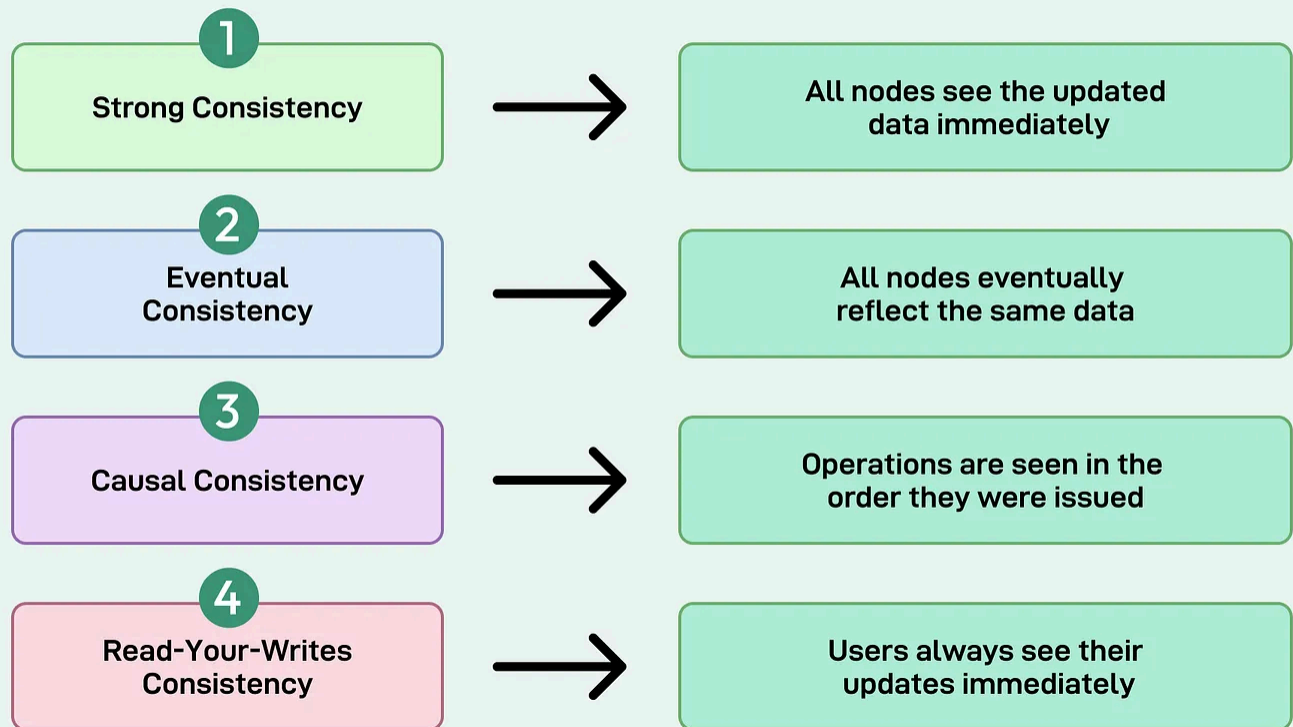
In a world where multiple services or databases operate independently, consistency ensures that no conflicting or outdated data is read or written. Without proper consistency mechanisms, users may receive incorrect information, leading to issues such as duplicate transactions, missing records, or unreliable system behavior.

Some user scenarios that can arise due to data consistency problems are as follows:

- A user places an order, but the system doesn't acknowledge the user, causing a duplicate order when the user places the order again.
- A user checks their bank balance, and it shows different amounts on the mobile app and the website.
- A user books a travel ticket, but due to an inconsistency, two people are assigned the same seat.

Different data consistency models determine how data updates are synchronized across services. Each model has trade-offs between accuracy, speed, and scalability. Let's look at them in more detail.

Different Types of Consistency Models



1 - Strong Consistency

It ensures that all nodes see the same updated data immediately after a transaction committed. Every read operation returns the most recent write.

For example, when we transfer money from Account A to Account B, both accounts must reflect the updated balance immediately.

A strong consistency model is ideal for financial transactions and inventory management in an e-commerce platform. This is to prevent discrepancies in account balances and make sure that orders are allowed only for available items.

The main challenges with strong consistency are as follows:

- **Performance Overhead:** Requires synchronization across multiple services, causing potential delays.

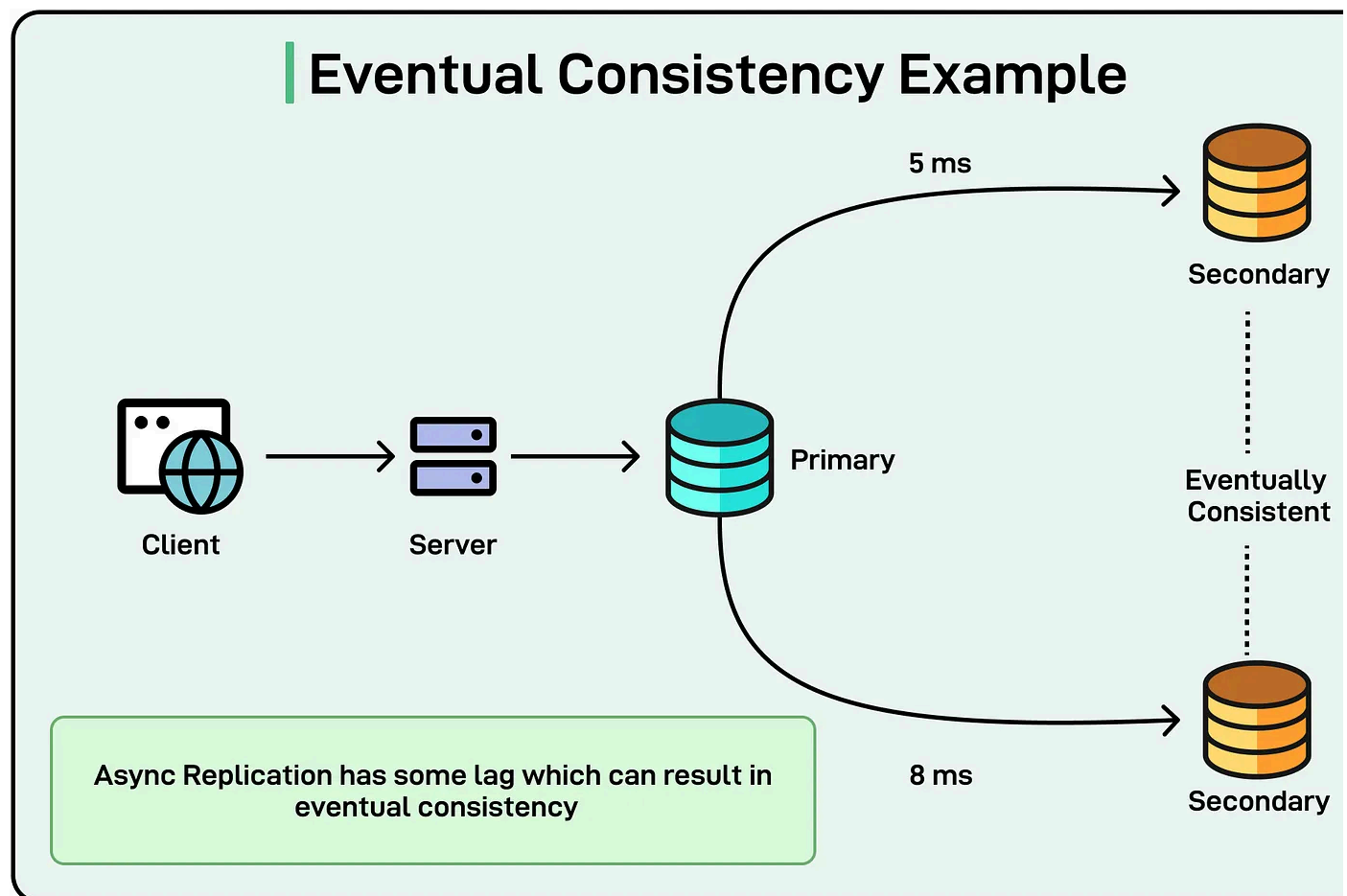
- **Reduced availability:** If one service goes down, the entire system may stall while waiting for updates.

2 - Eventual Consistency

This model guarantees that all nodes will eventually reflect the same data, but not immediately. Updates are propagated asynchronously across services. However, there may be a temporary period where different nodes in the database have different data.

For example, when we post a new status on our social media, it might take a few seconds for the connections to see the status on their social media feeds.

See the diagram below that shows eventual consistency in a primary-secondary database setup.



An eventual consistency model is ideal for scenarios where a slight delay is acceptable. However, the main challenges are as follows:

- **Temporary data discrepancies:** Users may see outdated data for a short time.
- **Conflict resolution:** In some cases, different services may process conflicting updates.

3 - Causal Consistency

This model guarantees that operations are seen in the order they were issued, thereby supporting cause-and-effect relationships. Unlike strong consistency, it doesn't ensure global synchronization but preserves logical dependencies.

For example, in a messaging application, if we send two messages (A and B), the recipient should see them in that order, even if they are delivered at different times.

Causal consistency is important for use cases like chat applications and collaborative editing tools. The main challenges are as follows:

- **Implementation Complexity:** Requires tracking dependencies between different operations.
- **Higher Latency:** Ensuring order across services can slow down performance.

4 - Read-Your-Writes Consistency

This model guarantees that users always see their updates immediately. Other users might still see outdated data, but the latest data is always visible to the person who made the change.

For example, if we update our social media profile, we should immediately see the update, even if other users still see the old details.

This consistency model is ideal for use cases like profiles, dashboards, shopping carts, and wishlist updates. The key challenge is to manage inconsistencies for other users when updates have not been fully propagated.

Choosing the Right Consistency Model

Here's a quick recap of when we can consider using a particular consistency model:

- If accuracy is critical, use strong consistency (for example, banking, and inventory management).
- If speed and availability matter more than real-time accuracy, use eventual consistency (for example, social media, and analytics dashboards).
- If ordering is important, use causal consistency (for example, chat applications).
- If users should see their updates immediately, use read-your-writes consistency (for example, shopping carts, and profile settings).

Strategies for Ensuring Data Consistency

Let us look at important strategies to ensure data consistency in a microservices architecture.

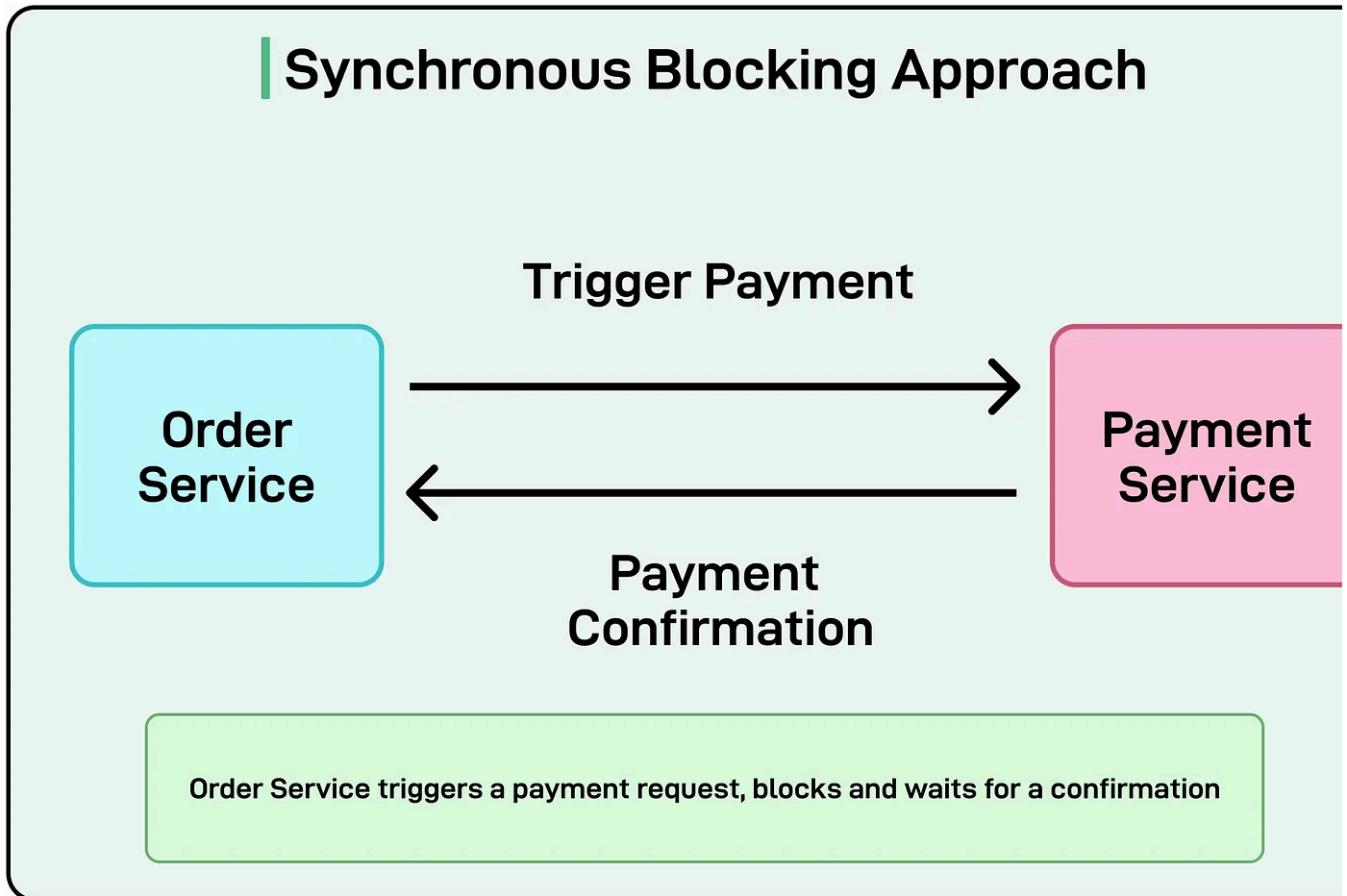
1 - Synchronous vs Asynchronous Communication

Synchronous communication occurs when a service sends a request to another service and waits for a response before proceeding. This typically follows a request-response pattern using protocols such as HTTP (REST) or gRPC.

Since the caller waits for a response, data updates happen in real-time. Transactions across services are processed in a deterministic order and errors (or failures) are immediately detected and handled to have a more predictable system state.

Here's a simple example of this approach in the context of processing payments:

- A customer places an order.
- The order service calls the payment service via an API.
- The payment service processes the transaction and confirms it.
- The order is then finalized only after a successful response.



Some challenges with this approach are as follows:

- **Increased Latency:** If a service takes longer to respond, the entire request chain is delayed.
- **Reduced Availability:** If one service is down, dependent services cannot proceed. This can result in a system-wide failure.
- **Tight Coupling:** Services become interdependent, making deployments and changes more complex.

Asynchronous communication occurs when a service sends a request and does not wait for an immediate response. Instead, it is free to process other tasks.

Since updates occur in multiple stages, different services might have temporary discrepancies. If a service fails to process a message immediately, it may take seconds or minutes for data to become fully consistent.

Here's a simple example from an e-commerce context that benefits from asynchronous approach:

- A customer places an order.
- The order service sends a message to a message queue (Kafka or RabbitMQ).
- The email service picks up the message and sends a confirmation email.
- Even if the email service is temporarily down, the message remains in the queue and is processed later.

Asynchronous Communication



2 - Choreography and Orchestration

Microservices often operate with independent databases, making distributed transactions challenging. This is where choreography and orchestration-based approaches can help establish data consistency.

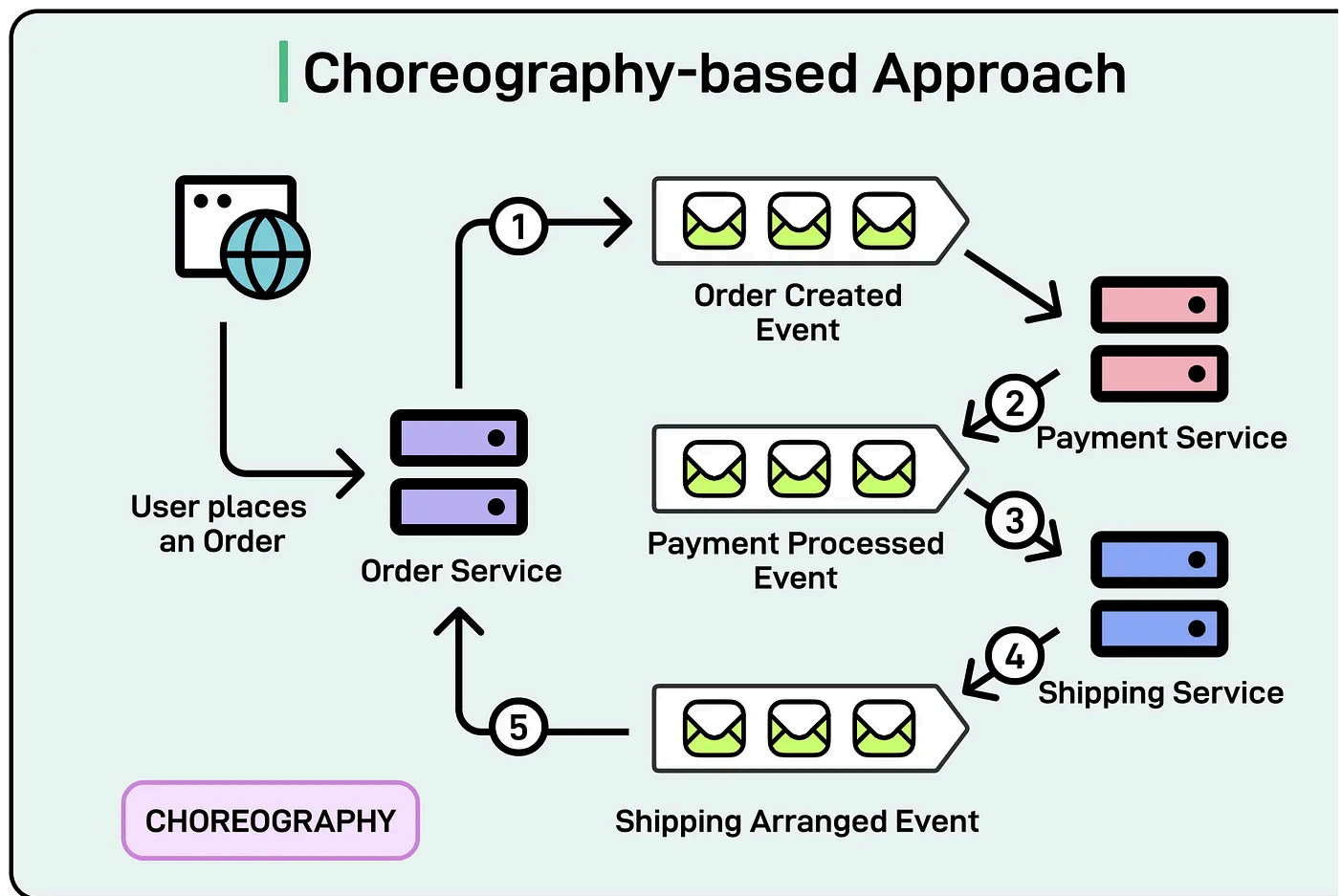
In a choreography-based approach, microservices communicate through events without a central controller. Each service listens for specific events and performs its task upon receiving a relevant event. If a service fails, it emits a compensation event to roll back prior changes.

Here's one example of the choreography approach in an order processing workflow

- The Order Service creates an order and emits an OrderCreated event.

- The Payment Service listens to OrderCreated, processes the payment, and emit PaymentProcessed.
- The Shipping Service prepares for delivery and listens to PaymentProcessed to arrange shipping.
- If payment fails, the Payment Service emits PaymentFailed. The Order Service listens and triggers a compensating transaction, changing the order status to “CANCELLED”.

See the diagram below that shows the choreograph-based approach:

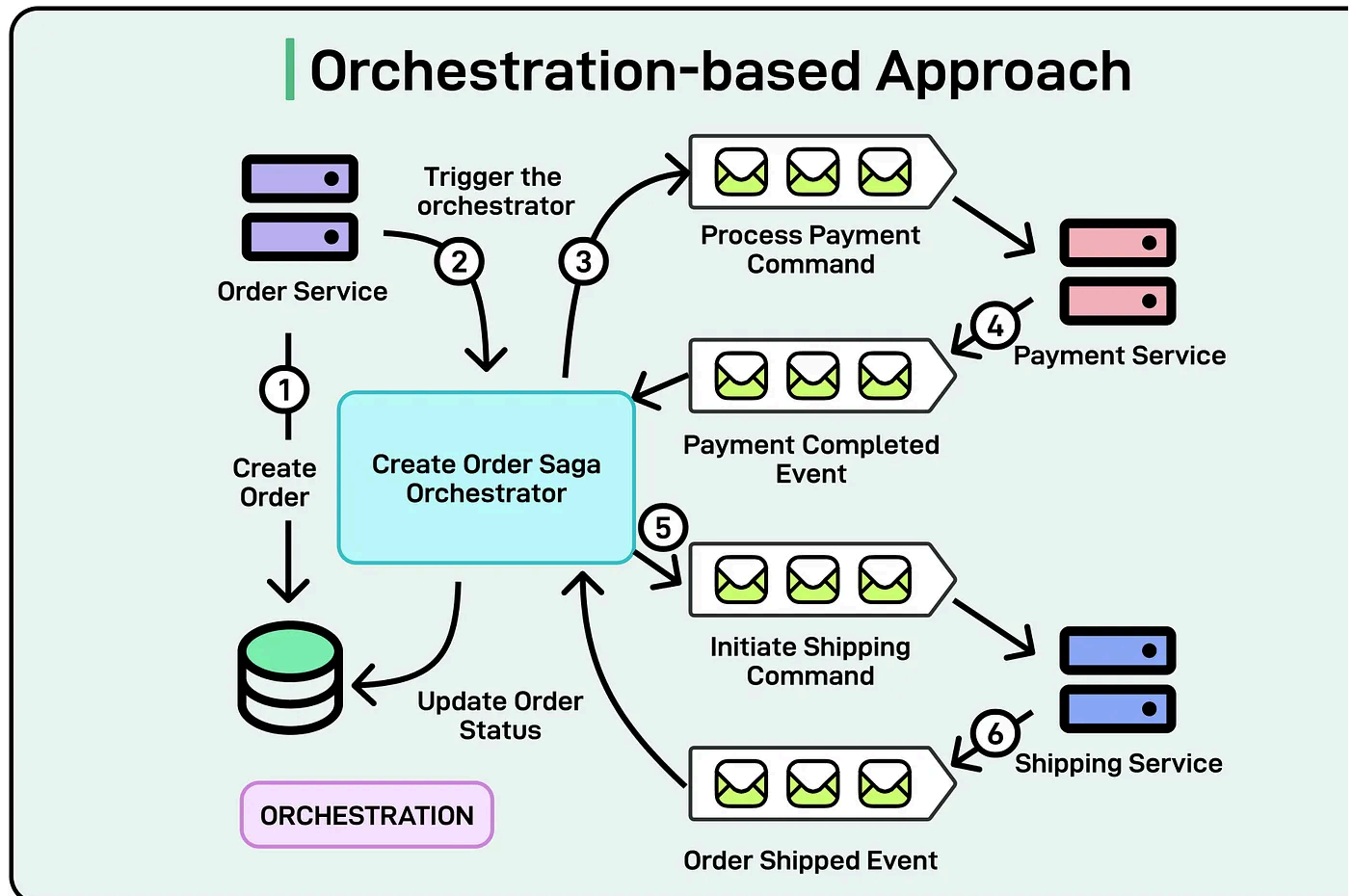


The second approach is orchestration. In an orchestration-based approach, a central orchestrator manages the sequence of service calls. The orchestrator explicitly calls each service and handles failures by invoking compensating actions.

Here's an example of the same:

- The Orchestrator receives an order request.
- It calls the Payment Service to process the payment.
- If successful, it calls the Shipping Service to initiate delivery.
- If the inventory update fails, the orchestrator calls the Payment Service to refund the transaction. It also calls the Order Service to mark the order as "CANCELLED".

See the diagram below that shows an example of an orchestration-based approach.



3 - Event-Driven Architecture

Microservices often rely on event-driven architectures to maintain eventual consistency while allowing services to remain loosely coupled and independently scalable.

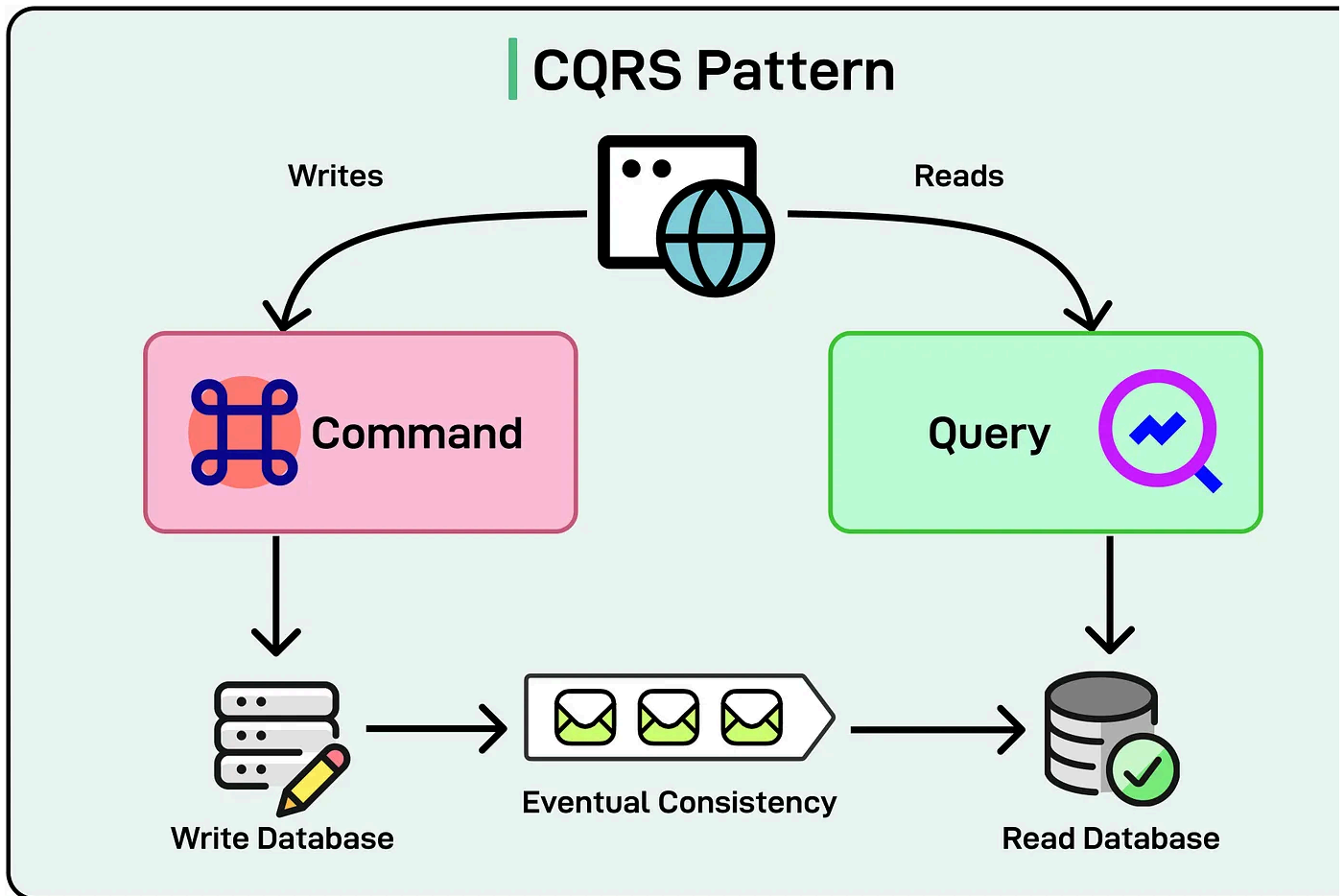
In an event-driven architecture, services emit events whenever state changes occur. Other services listen to these events and update their state accordingly, rather than making direct synchronous API calls.

A typical event-driven approach supports the following features:

- Instead of blocking API calls, services communicate through events, reducing dependencies and improving scalability.
- Services can process events at different times, ensuring they eventually reflect the correct state.
- If a service is down, events can be queued and processed later, preventing data loss.
- Event-driven systems handle high throughput since services operate independently.

CQRS is one approach that can use the event-driven approach to separate the write operations (commands) from the read operations (queries).

In this approach, the command model processes write operations and the query model is optimized for read operations to fetch data quickly. Since reads and writes are handled separately, scalability improves without affecting data consistency in a big way. Of course, the read database is eventually consistent with the write database.



4 - Database-Level Solutions

Some microservices architectures use distributed databases that provide built-in consistency mechanisms. Modern distributed databases can manage data across multiple nodes or data centers while ensuring consistency, availability, and fault tolerance.

Some key mechanisms through which distributed databases support these features as follows:

- **Global Transactions:** Distributed databases can support multi-region transactions that comply with ACID properties.
- **Consensus Protocols:** They use protocols like Paxos and Raft to ensure a consistent state across replicas.

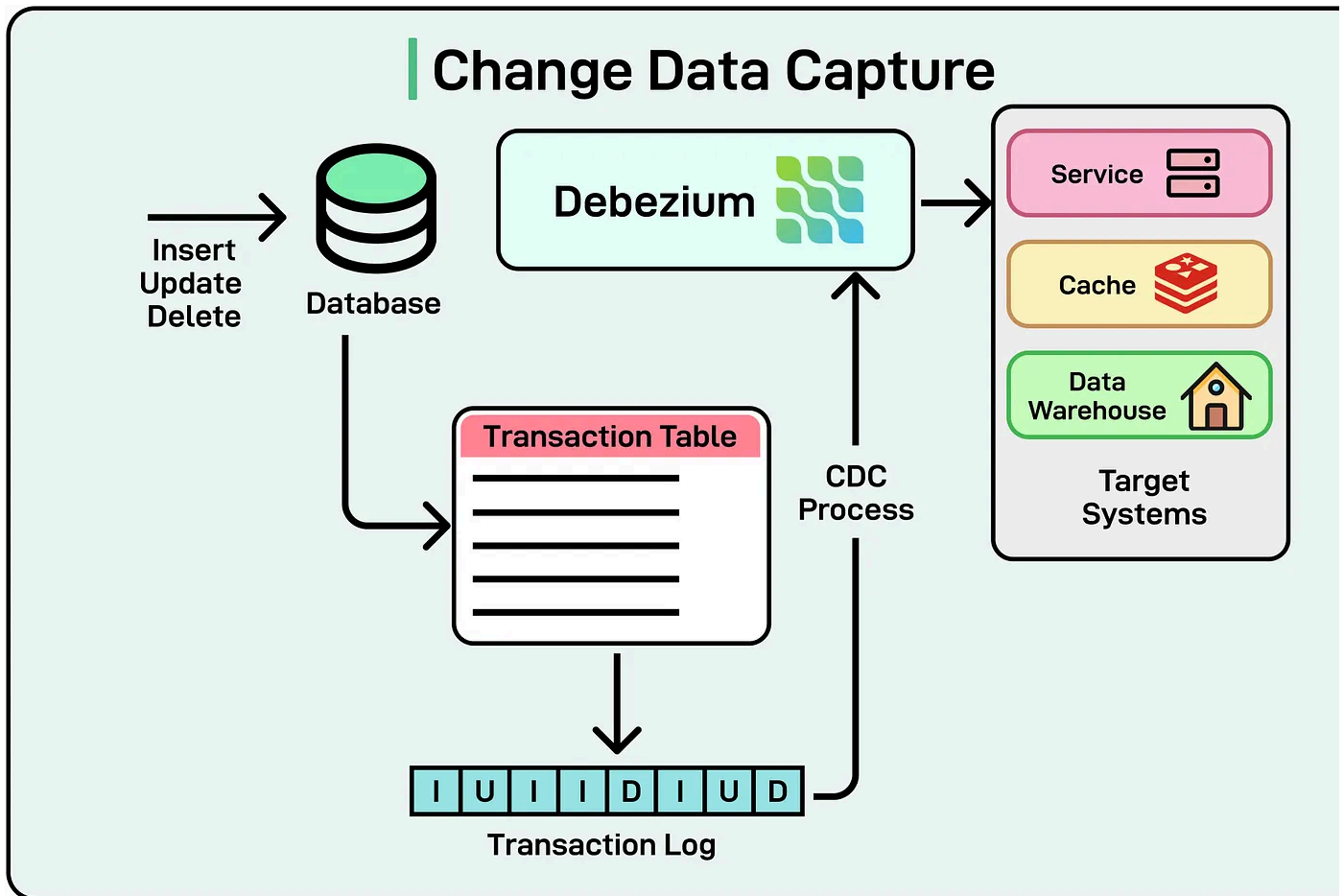
- **Strong vs. Eventual Consistency:** Many databases allow developers to choose between strong and eventual consistency based on business needs.

For example, Google Spanner uses a TrueTime API for globally distributed transactions that are executed in a globally ordered manner. Similarly, CockroachDB uses the Raft consensus algorithm to replicate data consistently across nodes.

Change Data Capture (CDC) is another database-level approach where changes to a database are tracked and published as events.

In CDC, the database maintains a change log (for example, MySQL binlog or PostgreSQL WAL). A CDC tool like Debezium monitors the logs and streams changes to other services. This allows microservices to stay eventually consistent by processing changes in near real-time.

See the diagram below that shows how CDC works in general:



Best Practices for Data Consistency

Let us look at some best practices to consider while designing for data consistency microservices.

1 - Choose the Right Consistency Model

Choosing between strong consistency and eventual consistency depends on the use case.

- Strong consistency ensures all services see the latest data immediately but can slow down performance. It is critical in banking and financial systems, where transactions must be consistent.
- Eventual consistency allows temporary delays in data updates but improves scalability. It works well for social media feeds, recommendation systems, and

analytics dashboards where real-time accuracy is not essential.

2 - Handling Failures Gracefully

Failures are inevitable in microservices architecture, so handling them effectively is key. Some techniques are as follows:

- Retry mechanisms ensure transient failures (for example, network timeouts) do not result in lost transactions.
- The circuit breaker pattern prevents cascading failures by stopping requests to failing services and allowing recovery time.
- Dead Letter Queues (DLQs) store messages that could not be processed successfully, preventing data loss while allowing later analysis or reprocessing.

3 - Versioning and Schema Evolution

Updating databases and APIs without breaking microservices is crucial.

Schema versioning ensures backward compatibility so services can handle both old and new data formats. Feature toggles allow rolling out database changes gradually reducing risks associated with instant updates.

4 - Observability and Monitoring

Maintaining visibility into system behavior helps detect inconsistencies early.

Distributed tracing (Jaeger, OpenTelemetry) tracks how requests flow through microservices, helping debug inconsistencies. Log aggregation and monitoring tools (such as ELK Stack, and Prometheus) provide insights into system health and data anomalies.

Summary

In this article, we have looked at the various strategies to maintain data consistency between microservices.

Let's summarize the key learning points in brief:

- Microservices operate with independent databases, making data consistency challenging. However, ensuring consistency is crucial for reliability, user trust, and preventing financial or operational errors.
- Different types of consistency models can be explored while designing the services.
- Strong consistency ensures immediate accuracy but impacts performance.
- Eventual consistency allows temporary inconsistencies but improves scalability
- Causal consistency maintains event order across services.
- Read-your-writes consistency ensures users see their updates immediately.
- Multiple strategies can be used to facilitate data consistency across services.
- Synchronous communication provides strong consistency but can cause bottlenecks.
- Asynchronous messaging improves scalability but requires handling eventual consistency.
- Choreography and Orchestration ensure consistency by handling distributed transactions with compensating actions.
- An event-driven architecture enables eventual consistency through event source and CQRS.
- Change Data Capture (CDC) streams database changes to ensure real-time consistency.

References:

- [Replication in Cockroach DB](#)

- **Spanner: Google's Globally Distributed Database**



274 Likes • 12 Restacks

Discussion about this post

Comments Restacks

Write a comment...

**Bronson**  Feb 28, 2025

I have a question about the choreography approach. In the situation where a payment failed e emitted, is that to a separate queue that the Order Service listens to? I am just missing the lin between the Payment Service and the Order Service.

 LIKE (1)  REPLY

1 reply

**Eric Aska**  Mar 16, 2025

What tool do you use for creating your highly illustrative diagrams?

 LIKE  REPLY

1 more comment...

© 2026 ByteByteGo · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture